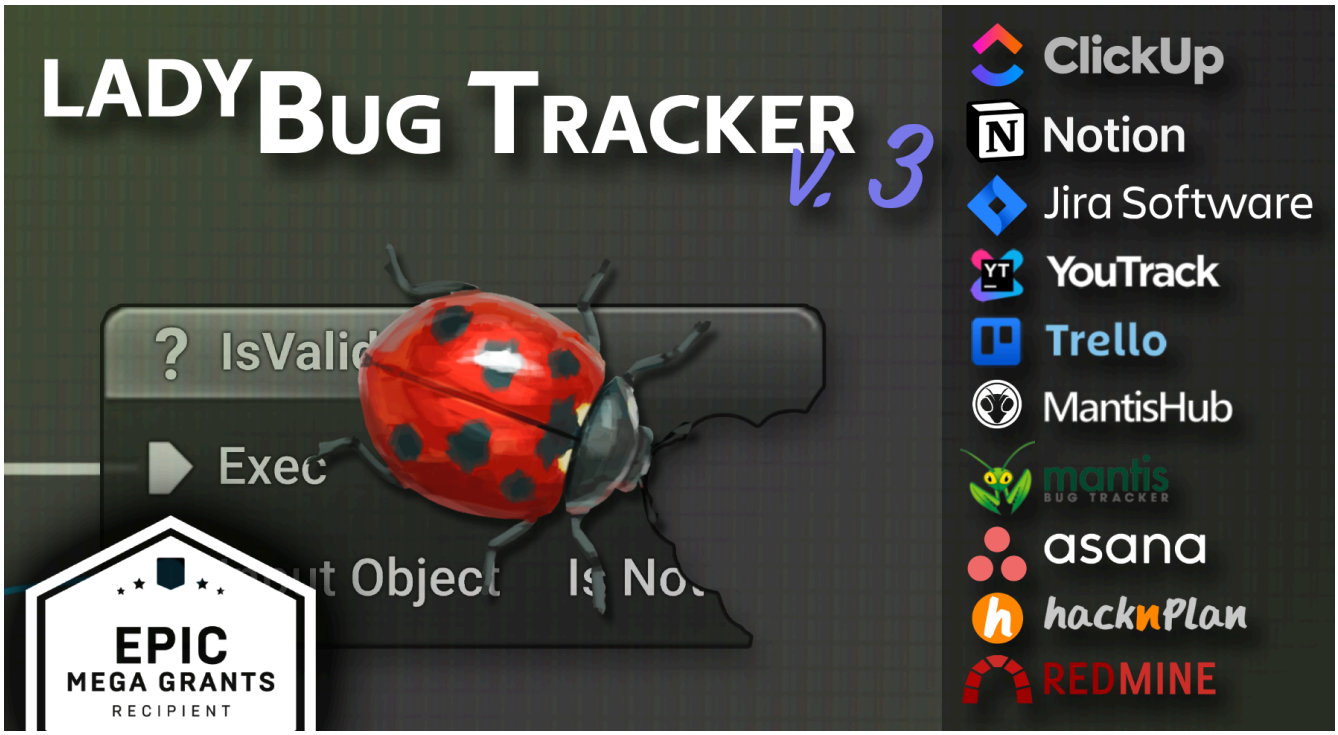
LadyBug Tracker



Author: **Tomasz Klin**
Version: **3.0.2**

Contents

Introduction

LadyBug Tracker is a plugin that integrates bug tracker systems with the UnrealEngine. Thanks to this plugin you will significantly increase the efficiency and convenience of your work. Forget about constant switching between the Unreal Engine editor and the browser with an open bug tracker page. Conveniently view and edit details of reported issues. Browse the issues directly in the level viewport. Report a bug straight from the editor and game. Create blueprints that automate reporting bugs for you.

Data Storage and Privacy

The LadyBug Tracker plugin does not store any data. All reports are sent directly to the databases of the integrated services (Jira, YouTrack, Mantis, Trello, Asana, HacknPlan, Redmine, ClickUp, Notion) using their respective APIs. Only you have access to this data, and it is never sent anywhere else. Additionally, no data is collected for statistical or diagnostic purposes.

Features list

- Browsing issues with a fast and powerful filter.
- 3D markers. View issue markers on a level.
- "Go to" button. One click to open the level in the location associated with the issue.
- Editing issues details.
- Reporting issues in the game.
- Reporting issues in the editor.
- Reporting crashes.
- Capturing screenshots and logs.
- Capturing level and camera position.
- Automatic filling fields in.
- Editing screenshots.
- Pending issues list. Complete new issues details before submitting.
- View issue details (general fields, custom fields, notes, attachments, history).
- Adding notes.
- Adding attachments

Supported Integrations

The ladyBug Tracker is designed to support many bug trackers systems.

YouTrack

- **Integration Documentation**
- https://www.jetbrains.com/youtrack/
- Free up to 10 users
- Project management tool
- Strongly recommended

Mantis Bug Tracker

- **Integration Documentation**
- https://www.mantisbt.org/
- Open source. Free to use
- Designed for bug tracking

MantisHub

- **Integration Documentation**
- https://www.mantishub.com/
- Hosted Mantis Bug Tracker version
- Designed for bug tracking

Trello

- **Integration Documentation**
- https://trello.com/
- Free up to 15 users
- Very popular software in game dev

Jira Software

- **Integration Documentation**
- https://www.atlassian.com/software/jira
- Free up to 10 users
- Project management tool

Jira Server

- This integration is in the beta stage and requires Jira Server 10.3.7 or higher
- Not supported labels
- Self-hosted paid version
- Project management tool popular in big companies

Asana

- **Integration Documentation**
- https://asana.com
- Free up to 10 users
- Project management tool

HacknPlan

- **Integration Documentation**
- https://hacknplan.com/
- Unlimited users

Redmine

- **Integration Documentation**
- https://www.redmine.org/
- Open source. Free to use
- Project management web application.

ClickUp

- **Integration Documentation**
- https://clickup.com/
- Free plan available
- Project management tool. On a free plan the Level and Camera values are stored in the task description instead of custom fields

Notion

- **Integration Documentation**
- https://www.notion.so/
- Free plan available
- Workspace and documentation tool.

Need other integration? Let me know

Integrations comparison

	YouTrack	Trello	Jira Software	Jira Server	Mantis	MantisHub	Asana	HacknPlan	Redmine	ClickUp	Notion
View issues	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Edit issues	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Report issues	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Custom fields	Yes	Yes[1]	Yes	Yes	Yes	Yes	Yes[1]	No[2]	Yes	Yes[1]	Yes
Sorting by columns	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Filtering	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
3D Markers	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
View attachments	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Add attachments	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Delete attachments	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No[2]	Yes
View comments	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Add comment	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Delete comment	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No[2]
View checklists (subtasks)	No[2]	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	Yes
Edit checklists (subtasks)	No[2]	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	Yes	Yes
Create project	No[2]	Yes	No	No	Yes	Yes	Yes	Yes	No	Yes	No[2]
View history	No	No	No	No	Yes	Yes	No	No	No	No	No
Free version users	10	15	10	-	unlimited	0	10	unlimited	unlimited	unlimited	unlimited[4]
Free version storagetotal/per file	30 GB / 500 MB	unlimited / 10 MB	2 GB / 2GB	-	unlimited / unlimited	0 / 0	unlimited / 25MB	500 MB / 5 MB	unlimited / unlimited	1 GB / 100 MB	unlimited / 5 MB
Free version projects	unlimited	10	unlimited	-	unlimited	0	unlimited	unlimited	unlimited	unlimited lists (max 5 Spaces)	unlimited

[1] Available in the paid version of the service

[2] Feature not provided by the service

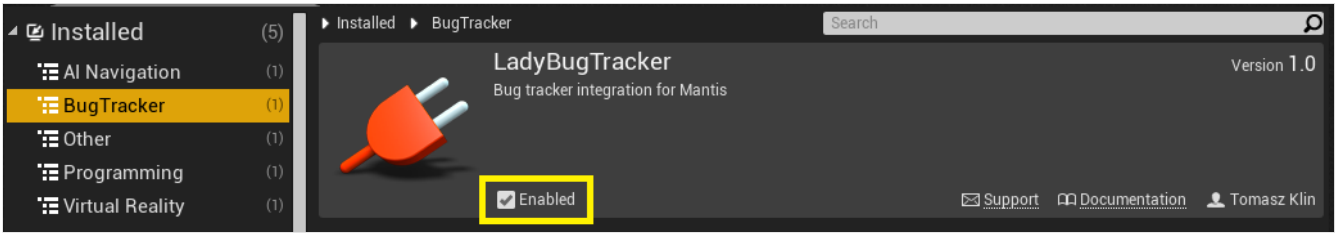
[3] Not yet implemented

[4] Notion applies a 1,000-block limit once a workspace has more than one member

Getting Started

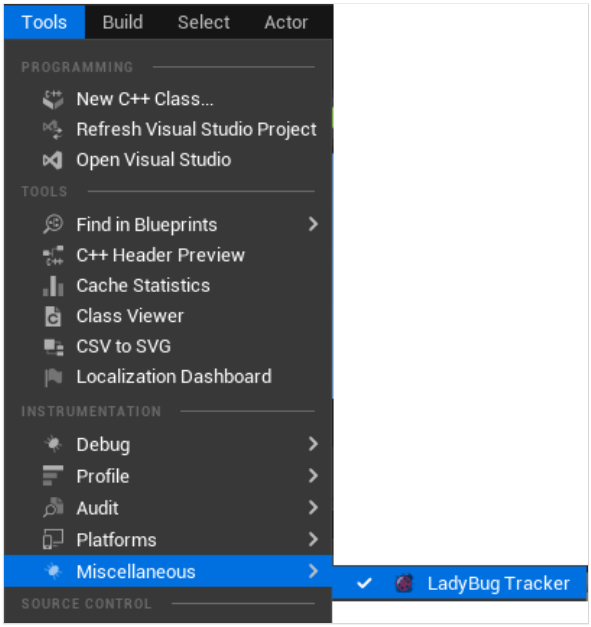
Enabling Plugin

In order to enable the plugin to go to Edit -> Plugins, select **Bug Tracker** category and check **Enabled** for LadyBug Tracker



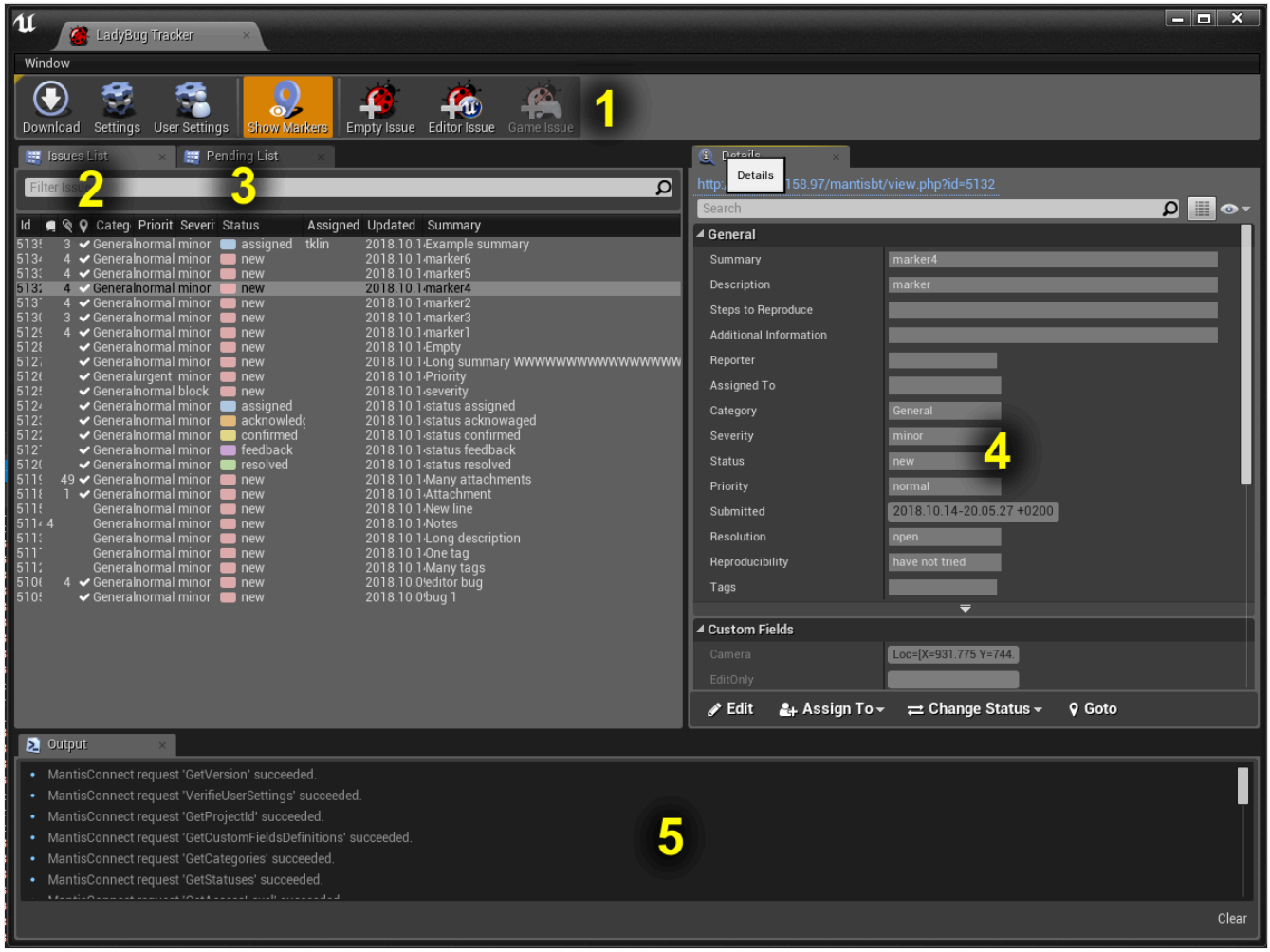
Opening Tool Window

LadyBug Tracker is available as a tab for Unreal Editor. The tab can be accessed from the Window menu.



Interface

The LadyBug Tracker user interface is made up of the following components:



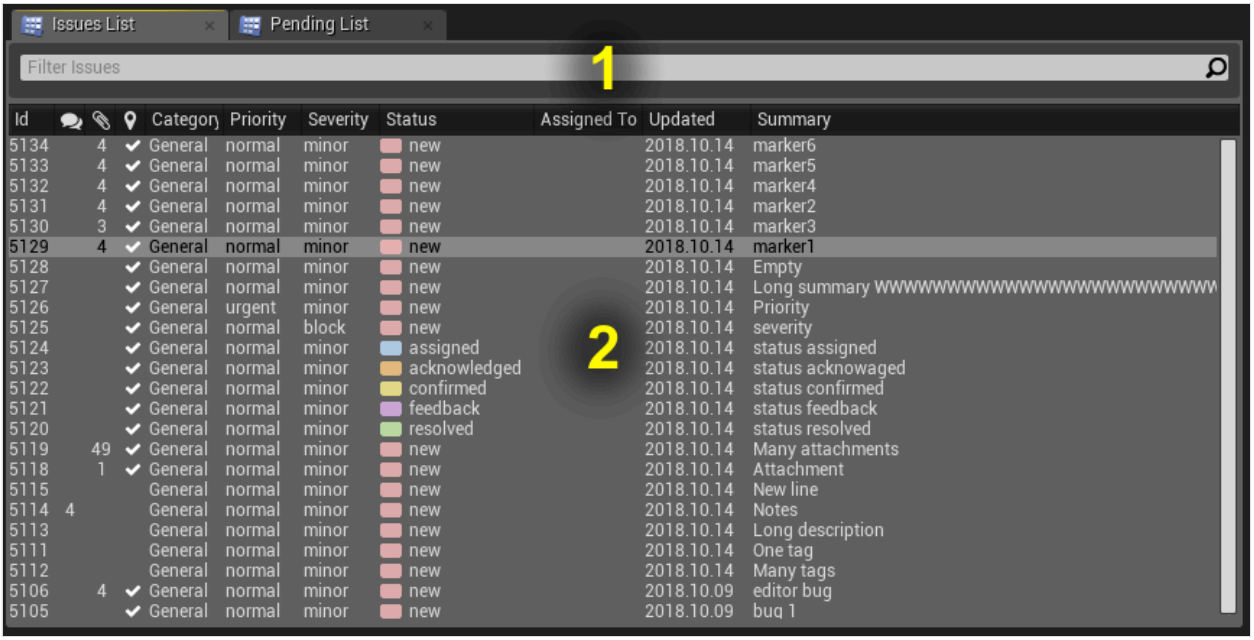
1. Toolbar
2. Issues List
3. Pending List
4. Details
5. Output

Toolbar

1. Download - download all issues from the server.
2. Settings - open the Project Settings window.
3. User Settings - open the login window.
4. Show Markers - show/hide markers in the Level Viewport (see 3D Markers)
5. Empty Issue - create a new issue on the pending list (see Reporting Bugs).

6. Editor Issue - create an issue from the editor (see Reporting Editor Issue).
7. Game Issue - create an issue from the game (see Reporting Game Issue).

Issues List



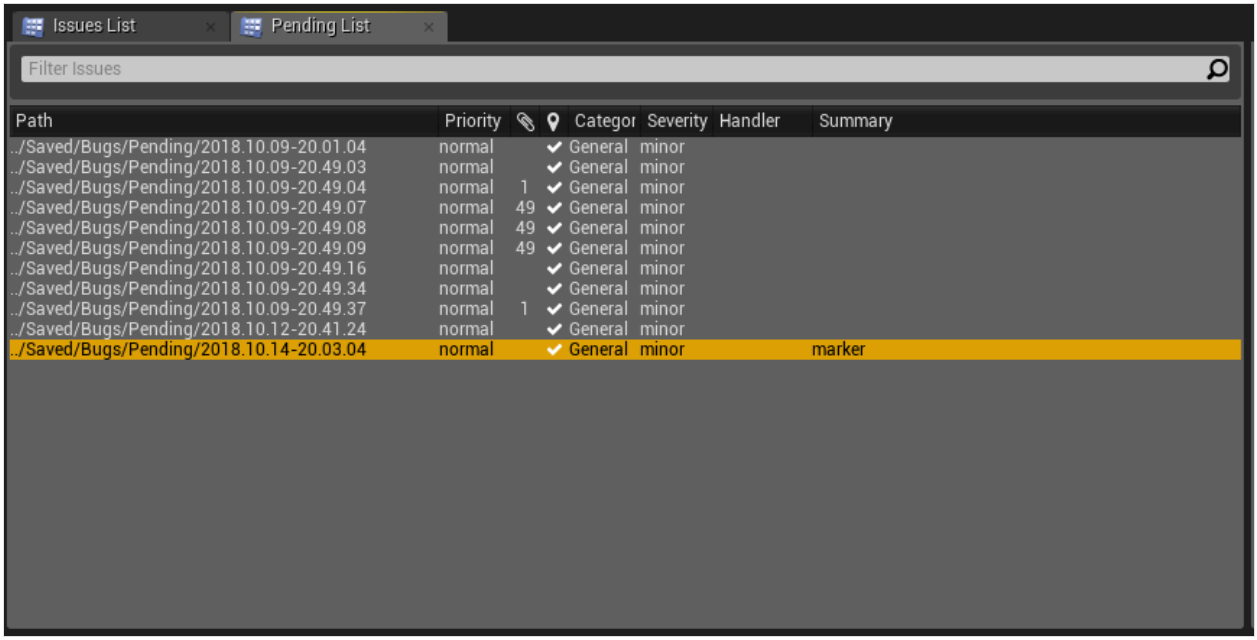
After downloading all bugs from the server, all issues are presented on this list (2). Issues from this list can be filtered to narrow the number of entries to the ones you are interested in (1). The Issue List filter enables you to search your issues using advanced syntax like in the Content Browser. The advanced search syntax supports sophisticated search queries, and allows searching by key-value pairs from issue properties. See the Advanced Search Syntax page for more information.

Example usage:

Filter Phrase	Description
hello	issues containing word 'hello' in the summary, description or additional information fields
...llo	issues containing a summary, description or an additional information ended with 'llo'
updated >= 2017	updated in 2017 or later
updated = 201712	updated in December of 2017
updated = 20171224	updated on 24 December of 2017
updated < 20171224.13	updated before 1 pm 24 December of 2017
status != resolved	all unresolved issues
severity > minor	severity greater than minor
assignedto = john_smith	assigned to john smith
submitted = 2018 & status = new	new issues submitted in 2018

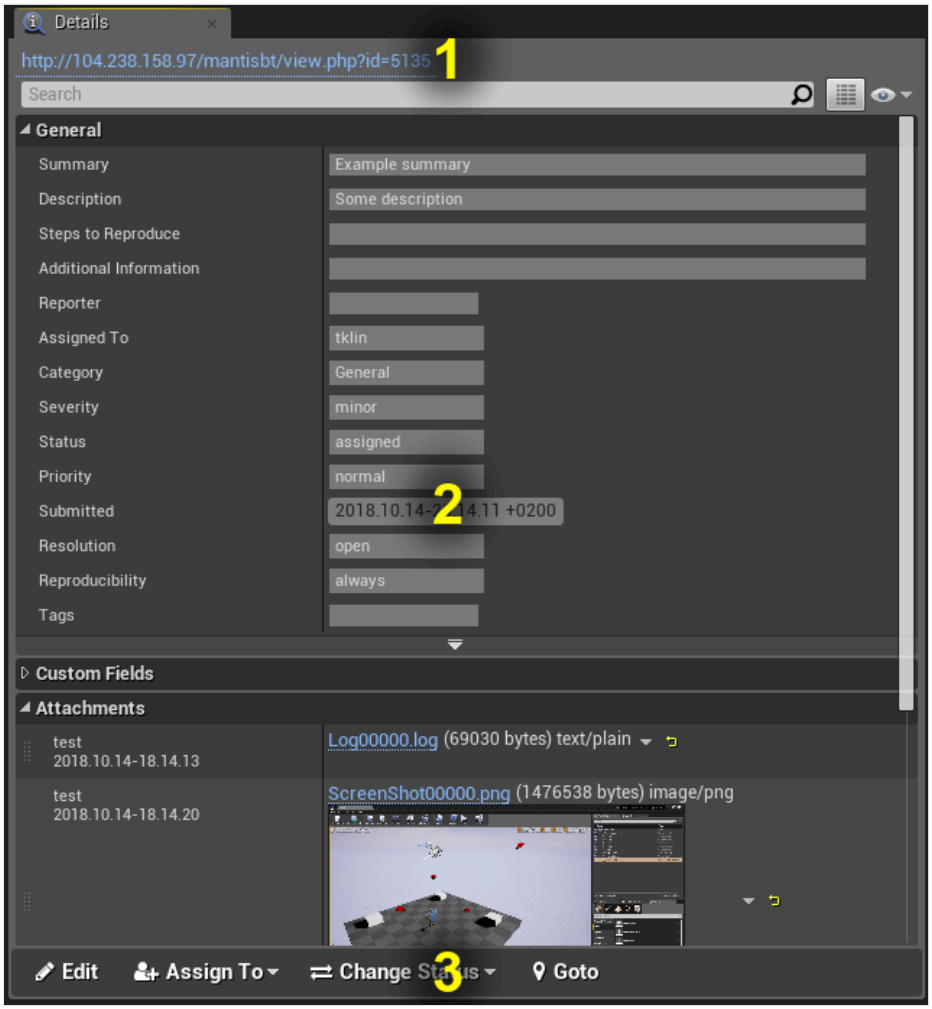
Pending List

The Pending List contains a list of pre-reported bugs that are expected to be filled in and submitted to the server (see Reporting Bugs). Similarly to Issues List, the Pending issues can be filtered.



Details Panel

Details panel presents all the information available for the selected bug.



1. A link that navigates to the selected issue in the web browser
2. Property list (see Property List)
3. Toolbar (Details Panel Toolbar)

Property List

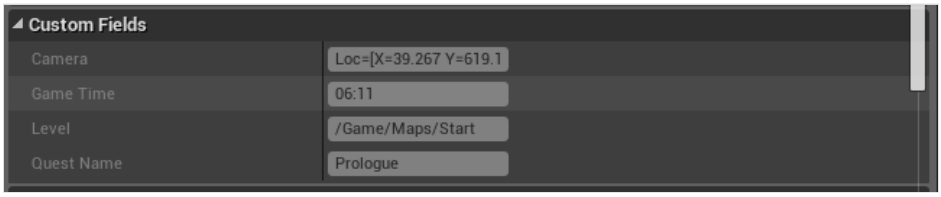
Contains all the most crucial information for the selected issue.

General

Standard properties are available by default in the Mantis.

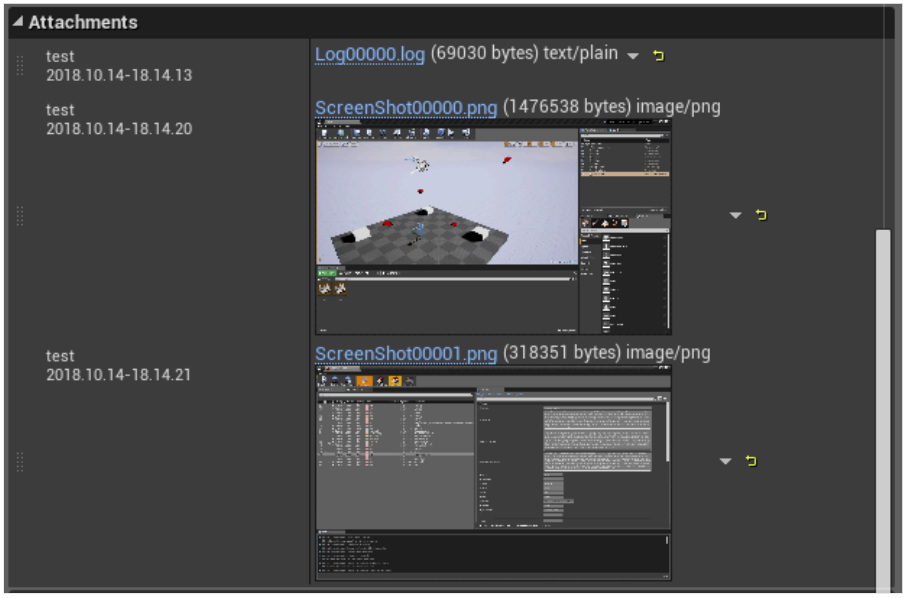
Custom Fields

User-defined fields. Custom fields are represented only as text. Validation is made on the server side during submission.



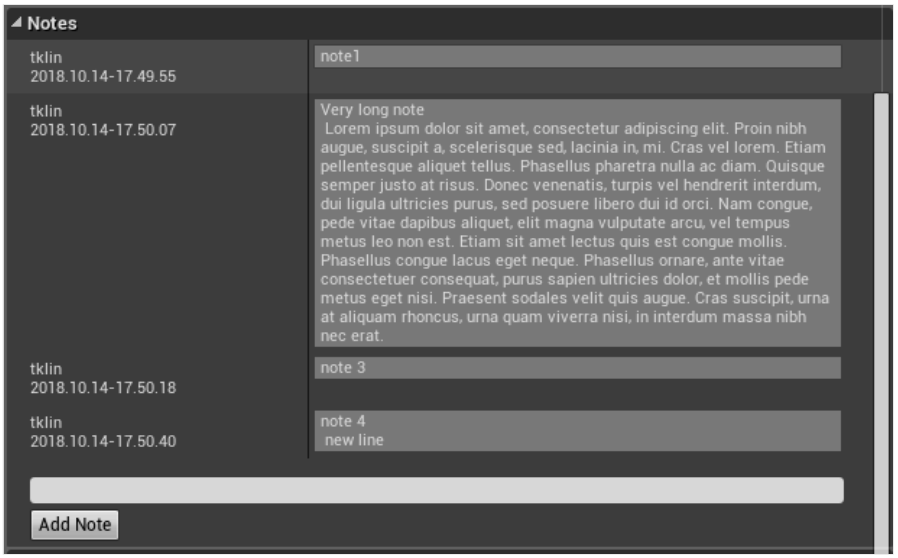
Attachments

The Attachments section presents a list of attachments. For images in PNG, BMP, and JPG formats, the plugin can display thumbnails.



Comments

The Comments section presents all notes added to the issue. Here you can also add new notes.



History (only mantis integration)

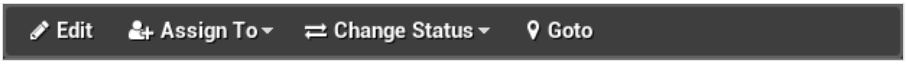
History of all activities corresponding to the issue.

Note: For now, history doesn't show readable values for enum types like category or priority. Instead, the numeric value is displayed.

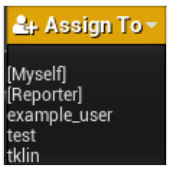
Details Panel Toolbar

Depending on the current mode the toolbar presents a different set of controls.

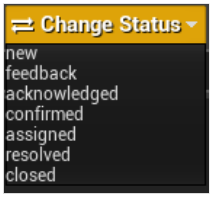
View Mode



1. Edit - Starts the edit mode.
2. Assign To - Quick assign the issue to a user.



1. Change Status - Quick change the issue status.



1. Goto - Open level in the location specified in the issue (see 3D Markers).

Edit Mode

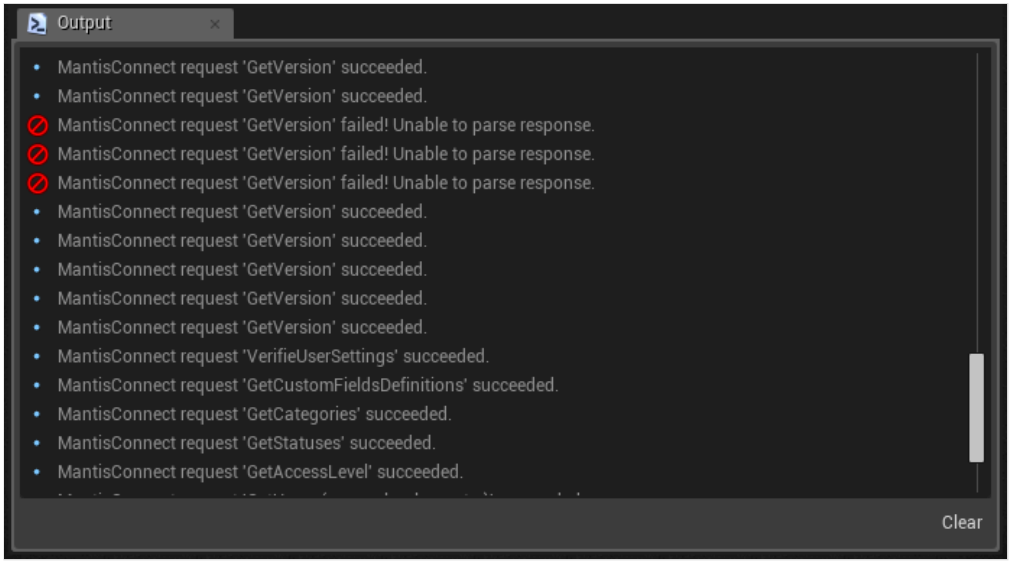
1. Cancel - Cancel editing. All changes will be discarded.
2. Update - Submit all changes to the server.

Report Mode

1. Discard - Delete the issue from the pending list.
2. Submit Issue - Submit the issue to the server.

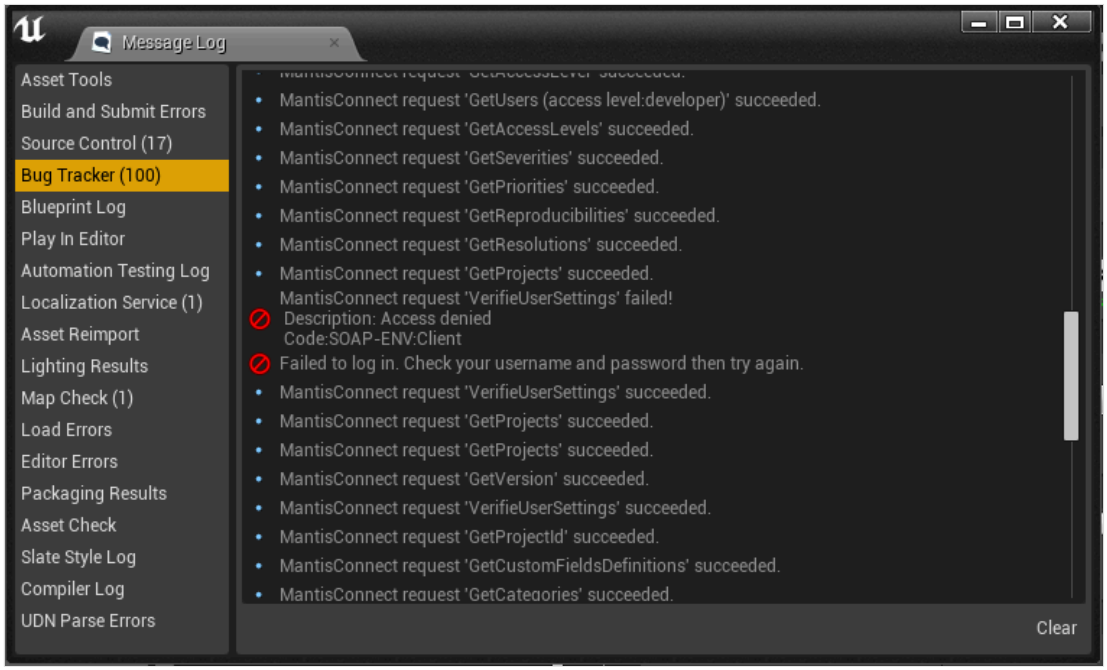
Output

The Output panel is the central hub for gathering the log output of plugin operations. In case of any troubles, this is the first place where you should look for causes.

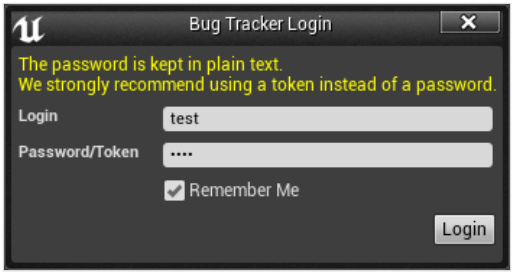


Message Log

The same logs that appear in the Output panel you can also find in the Message Log tab in the Bug Tracker section . You can increase logs verbose in the project settings in the section “Debug”



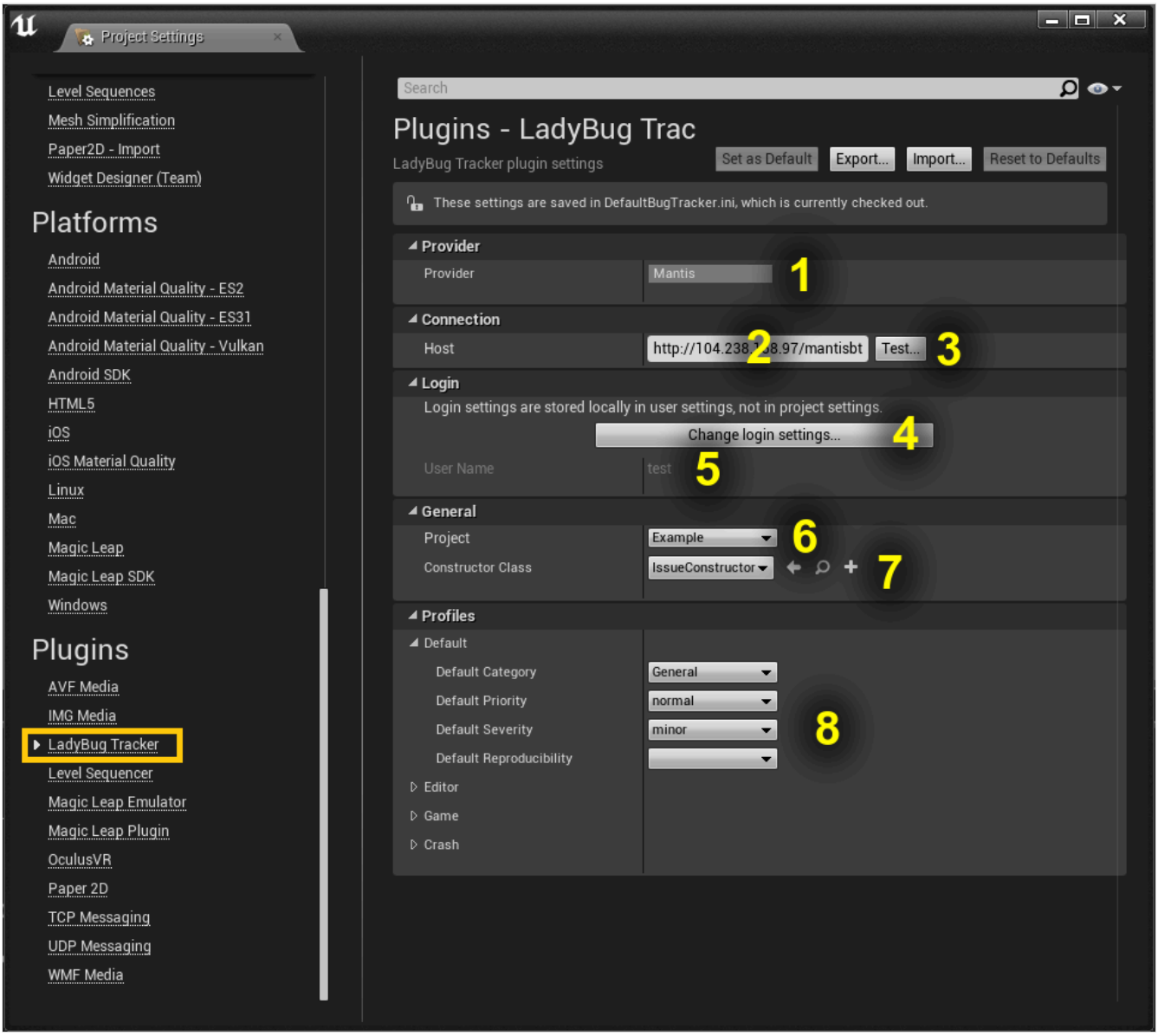
Login Dialog Box



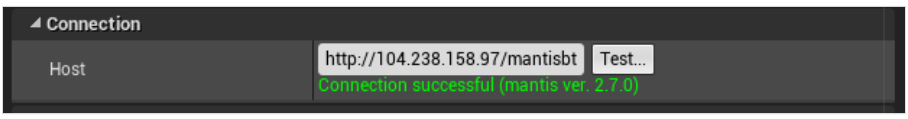
This window may be different dependent on chosen provider.

Project Settings

LadyBug Tracker settings can be edited in the Project Settings tab under the Plugins section. The settings are stored in **/Config/DefaultBugTracker.ini file** and can be shared by your team.



1. Provider - Bug tracker provider
2. Host - Address to bug tracker host.
3. Test - Test connection to the host.



1. Change Login Settings - Open login window (see Login Dialog Box).
2. User Name - Current logged-in user.
3. Project - Here you can select an available project for the logged-in user.

4. Constructor Class - Issue constructor class (see Issue Constructor)
5. Profiles - Default values for each of the issue profiles (see Profiles).

User Settings

User settings like preferences, login, and password are stored in **/Saved/Config/WindowsEditor/BugTrackerPerProjectUserSettings.ini** file**.**

Markers

When reporting bugs in the editor or game, the plugin automatically saves the level name, location, and orientation of the camera. Each bug that has this information is marked on the Issues List in a column with

a symbol.

Attention! This feature is available only if the Camera and Level fields are added in the custom fields.

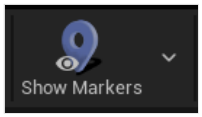
Goto

Thanks to the location information stored in an issue, we can jump very fast to the level in the location associated with the issue. To do this, just click the button

on Details Panel Toolbar.

3D Markers

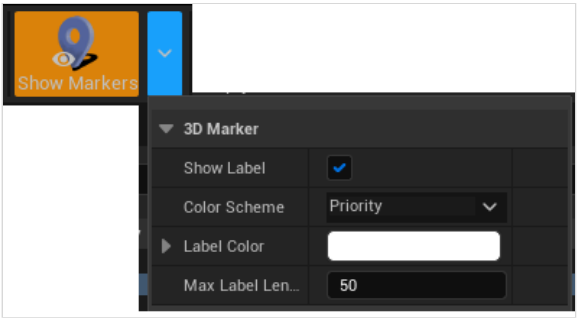
All issue markers associated with the currently opened level can be displayed in the level viewport. To enable this feature check



in Toolbar.

Selecting a marker on the level viewport opens the issue in Details panel. Double-click on a marker jumps to the marker's location.

3D markers labels can be adjusted from here:


Show Markers
3D Marker
Show Label
Color Scheme
Priority
Label Color
Max Label Len...
50

Reporting Bugs in Editor

The most powerful feature of this plugin is the very convenient and customizable reporting issues.

All reported bugs are sent to the pending list, thanks to which we can describe and submit an issue on the server at any convenient moment. For each newly added bug, a subdirectory is created in the ../Saved/Bugs/Pending/ directory. All files that are there, will be automatically added to the bug as attachments.

Reporting Editor Issue

Issues can be reported in two ways. From the Toolbar or using the keyboard shortcut after earlier assigning to some key. The bugs reported in the editor automatically include the log and screenshots of all open editor windows.

Reporting Game Issue

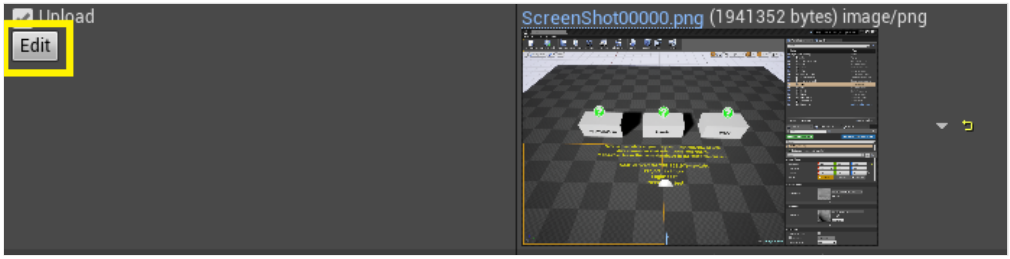
Bug reporting in the game can be done in three ways. From the Toolbar, from the console, or from a blueprint (see Triggering Reporting Bugs). To report a bug from the console, open the console in the game and fire the command **ReportBug bug_summary**. The bugs reported from the game will have attached in a screenshot and the log.

Reporting Crashes

Each crash that happened will also appear on the pending list, so it is very easy to report it. Bugs from crashes contain the log from the game, minidump, and other files saved by UnrealEngine's crash reporting tool.

Editing Screenshots

Before submitting an issue, all screenshots can be edited in a user-defined image editor e.g paint.



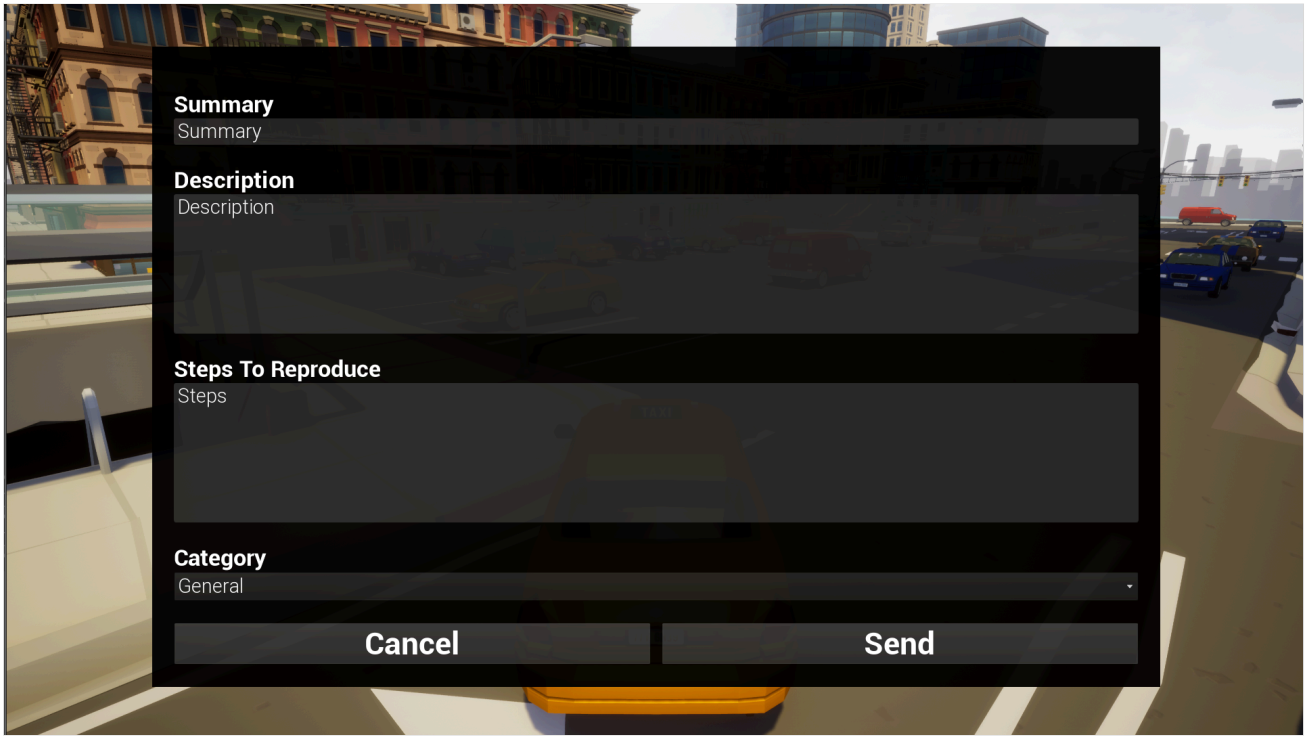
By checking **Upload** checkbox you can decide what files should be attached.

Reporting User Feedback

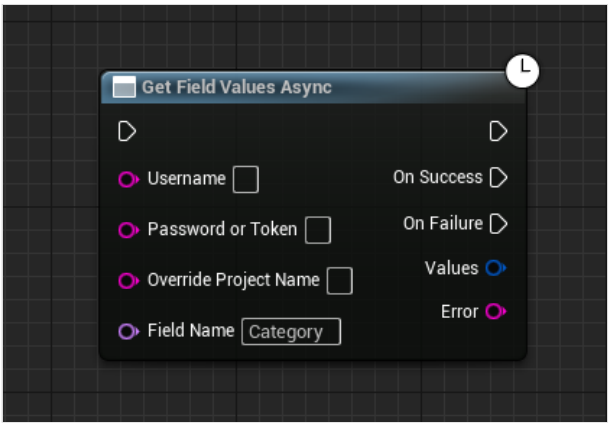
LadyBug Tracker allows to send feedback from users with attachments (logs/saves/screenshots) in the shipping game.

For user feedback submissions, we use authentication data different from those used in the editor. It is good practice to use a separate account created specifically for this purpose for reporting bugs. For security reasons, the account should have minimal permissions assigned.

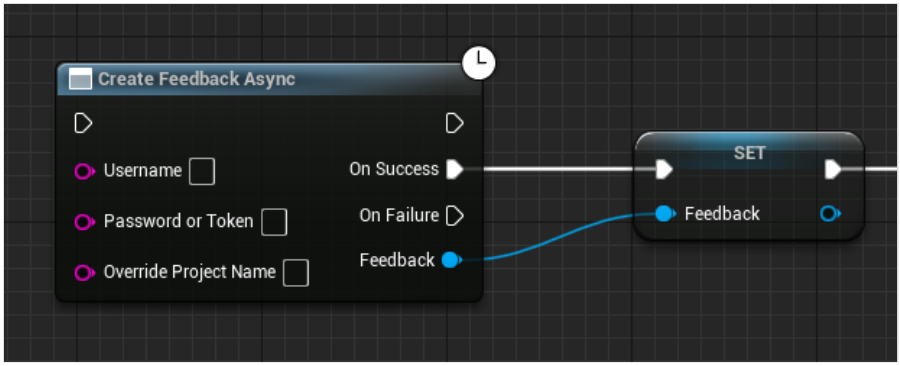
1. Create a feedback form that can look like the one provided with the plugin (see \LadyBugTracker\Content\UIExample\ReportForm).



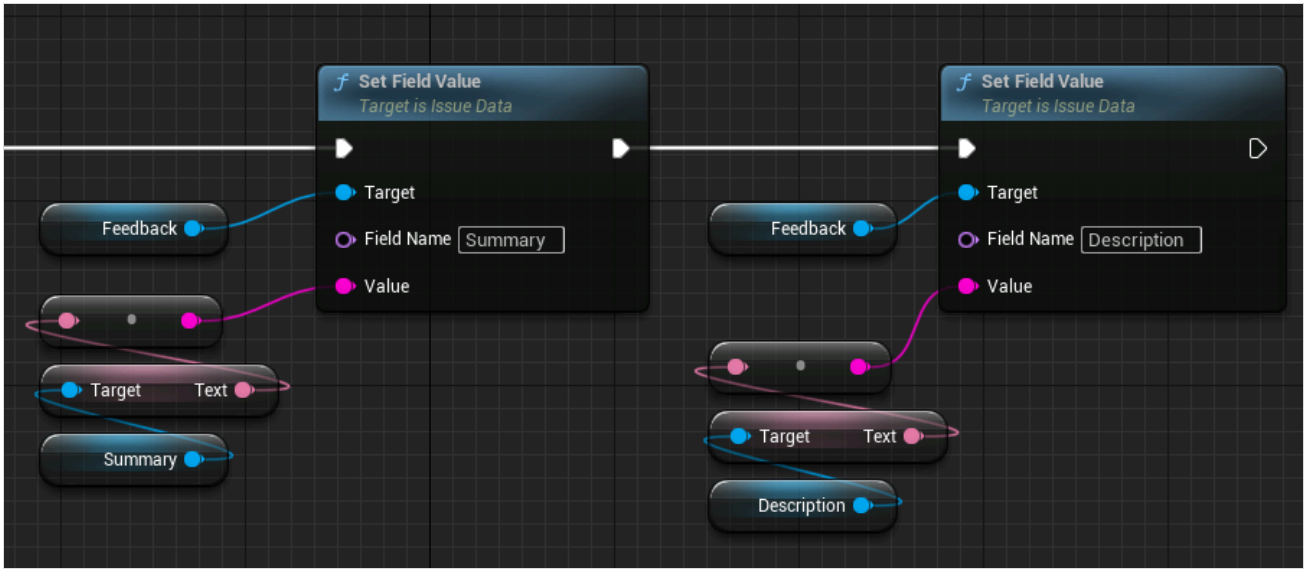
2. If you need to populate your UI with possible values for a given field use the **Get Field Values Async** node



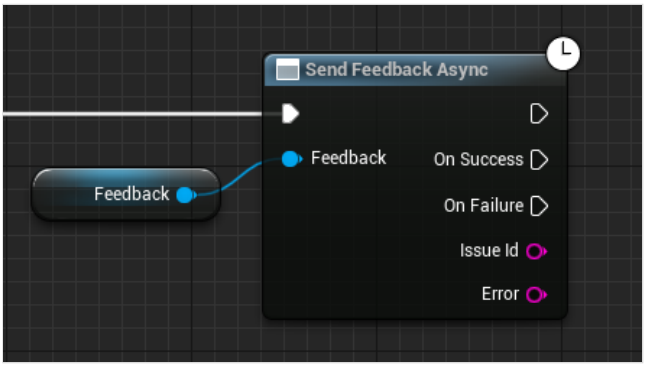
3. Create feedback object with **Create Feedback Async**



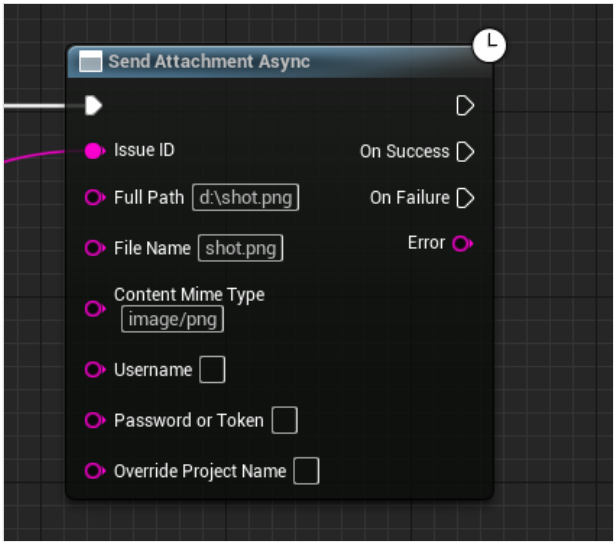
4. Set all data that you want to provide



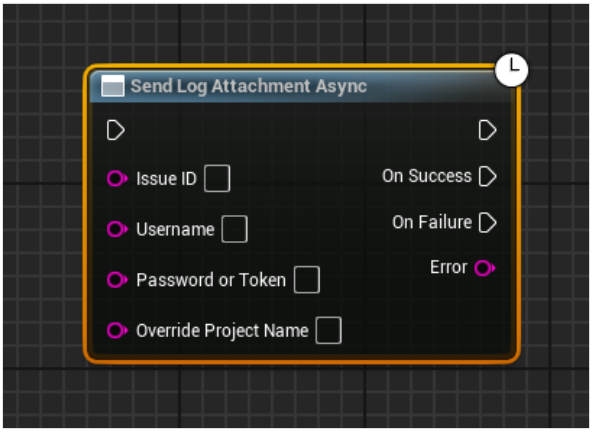
5. Send feedback to the server with **Send Feedback Async**



6. Send Feedback return Issue Id that can be used to upload to issue attachments with **Send Attachment Async**



7. To attach data the game already holds in memory - a save, a generated report, a screenshot you took yourself - use **Send Attachment From Memory Async** instead. It takes the same Issue Id, file name and mime type, and a byte array in place of the path.
8. To attach the game log use **Send Log Attachment Async**. It collects the log itself, so the Issue Id is all it needs.



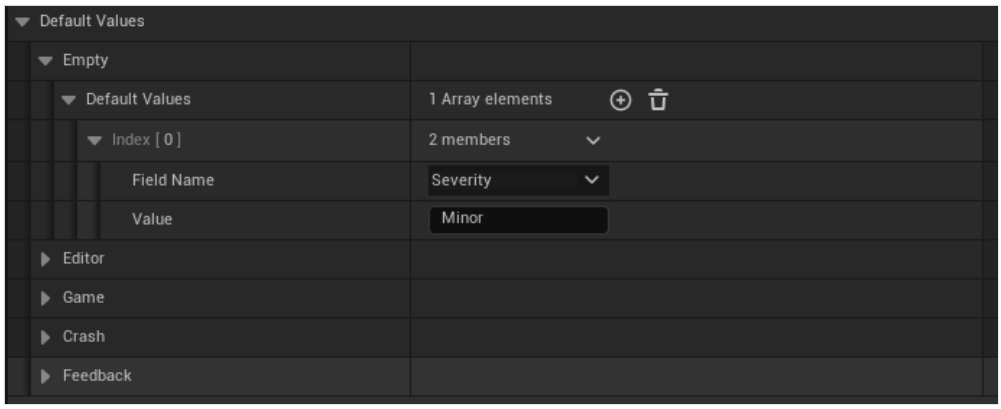
All the nodes used here have a synchronous counterpart; both variants are listed in Blueprint API.

Reporting Automation

To increase QA team efficiency, some of the tedious work can be done automatically for them.

Profiles

Profiles help to override default values for given fields dependent on a report issue mode. To override these default values open LadyBug Tracker Project Settings.

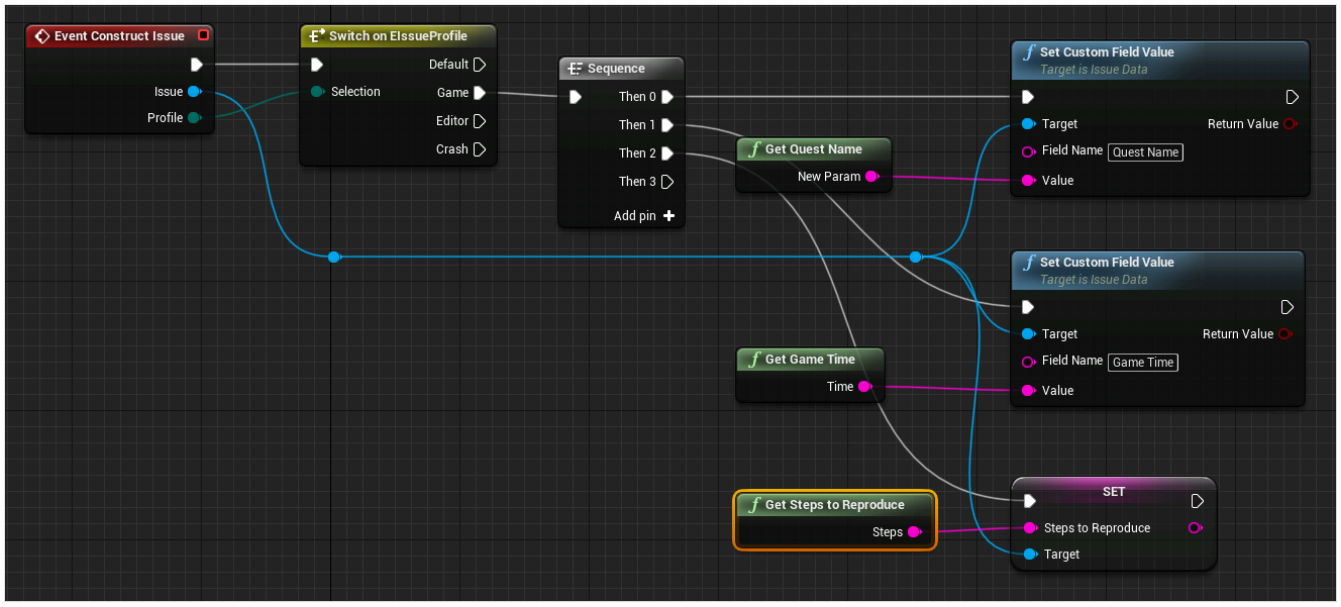


Issue Constructor

Issue Constructor is a blueprint class that can modify any issue parameters. E.g. you can add additional information that helps reproduce issue like current quest name or player character skills.

To create your own Issue Constructor you need to create a blueprint that inherits from **IssueConstructor** and overrides **Construct Issue** function. Function **Construct Issue** will be called for all bugs just after the issue will be reported.

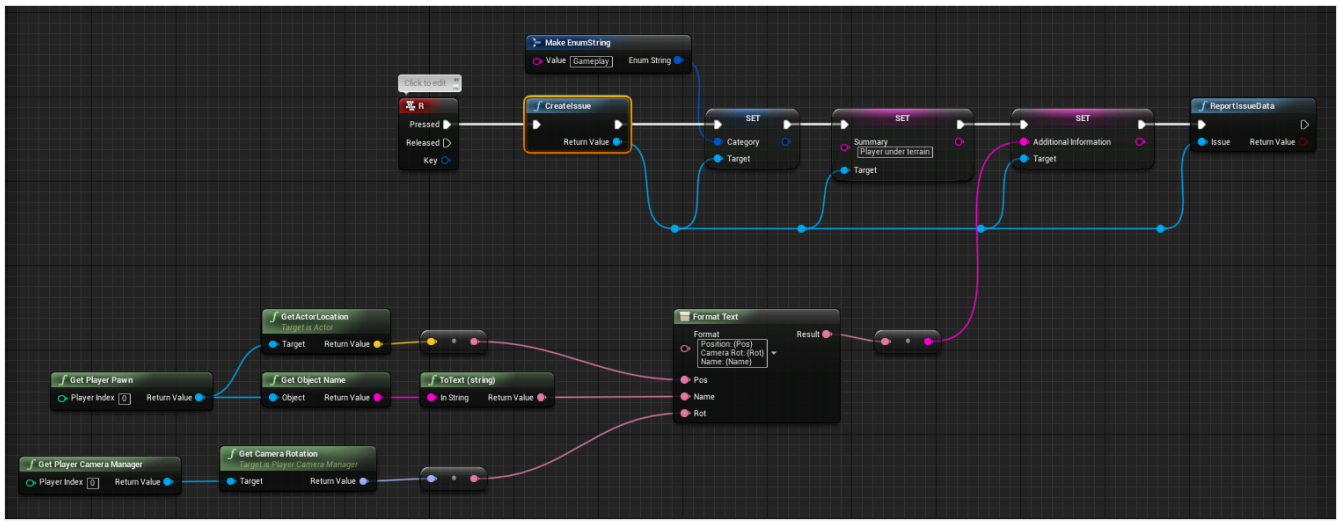
In the example below for all issues reported in the game, information about the current level, the game time and some steps to reproduce will be added.



To set your Issue Constructor, select an IssueConstructor class from the combo box in Project Settings.

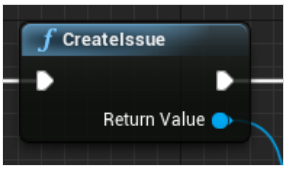
Triggering Reporting Bugs

You can also automatically create issues when some certain condition appears in the game. E.g. when the player falls under the terrain.

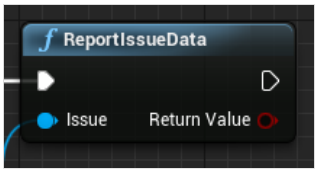


This reporting method allows you to add additional information like player position, colliding objects etc.

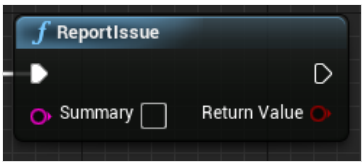
To report bug, call node **CreateIssue**. This node only creates Issue object that can be modified later.



After Issue object has been set up we must call ReportIssueData function. The reported issue will go to the Pending List.



You can also call **ReportIssue** which creates and reports an issue with a given summary.



Blueprint API

All the nodes the plugin adds are in the **Bug Tracker** category. Everything that talks to the server comes in two variants of the same node:

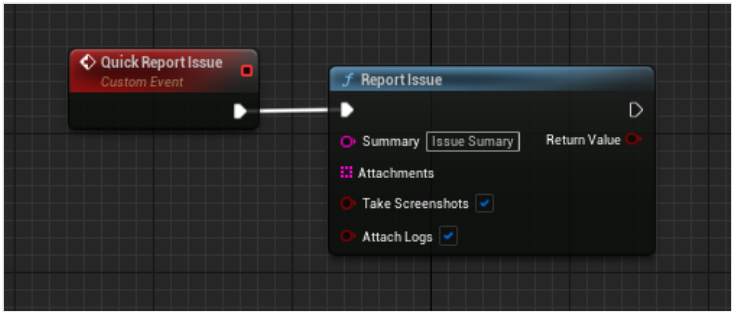
- **Synchronous** - the node returns immediately and reports the result through its **On Success** and **On Fail** delegate pins, which you bind to your own custom events. Examples 1 to 4 below are written this way.
- **Asynchronous** - the node name ends with **Async** and, in place of the delegates, it has **On Success** and **On Failure** execution pins that fire once the server answers. Nothing has to be bound by hand, so these are the easier ones to start with. Example 5 shows the same graph with them.

Node	Asynchronous variant	What it does
Create Issue	-	Creates an empty issue to fill in and hand to **Report Issue Data**.
Report Issue Data	-	Sends an issue created by **Create Issue** to the pending list.
Report Issue	-	Creates and reports an issue with a given summary in one step.
Create Feedback	**Create Feedback Async**	Creates the feedback object your form fills in.
Send Feedback	**Send Feedback Async**	Submits the feedback and returns the Issue Id.
Send Attachment	**Send Attachment Async**	Uploads a file from disk to that issue.
Send Attachment From Memory	**Send Attachment From Memory Async**	Uploads a byte array to that issue, with no file on disk.
Send Log Attachment	**Send Log Attachment Async**	Uploads the current game log to that issue.
Get Field Values	**Get Field Values Async**	Returns the possible values of a field, to populate your own UI.
Get Issues	**Get Issues Async**	Returns the issues of the project, so the game can list them itself.
Get Bug Tracker Name	-	Returns the name of the configured integration.

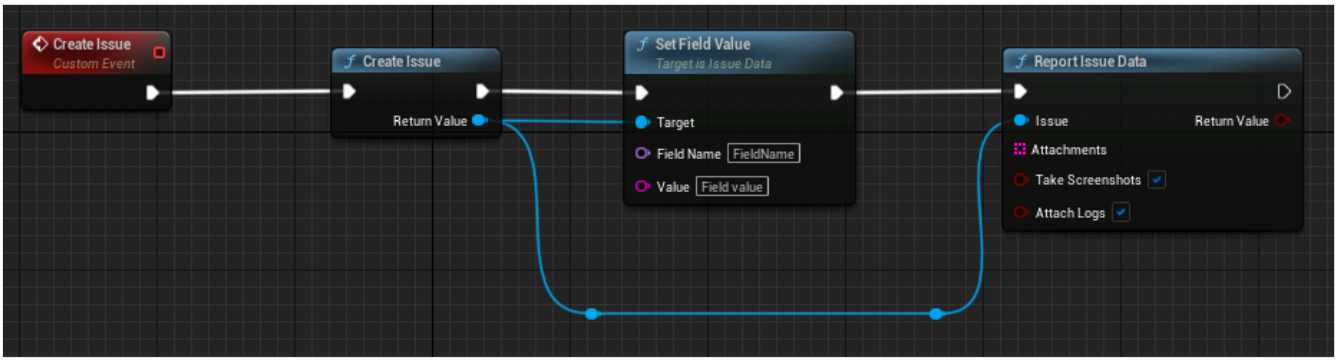
Where a node asks for **Username** and **Password or Token**, give it the separate reporting account described in Reporting User Feedback, never your own. Leave **Override Project Name** empty and the project from Project Settings is used.

Blueprints example

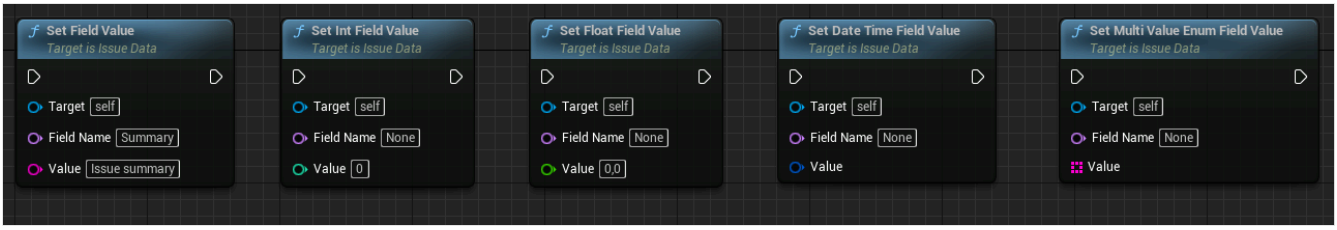
Example 1 - Quick report issue



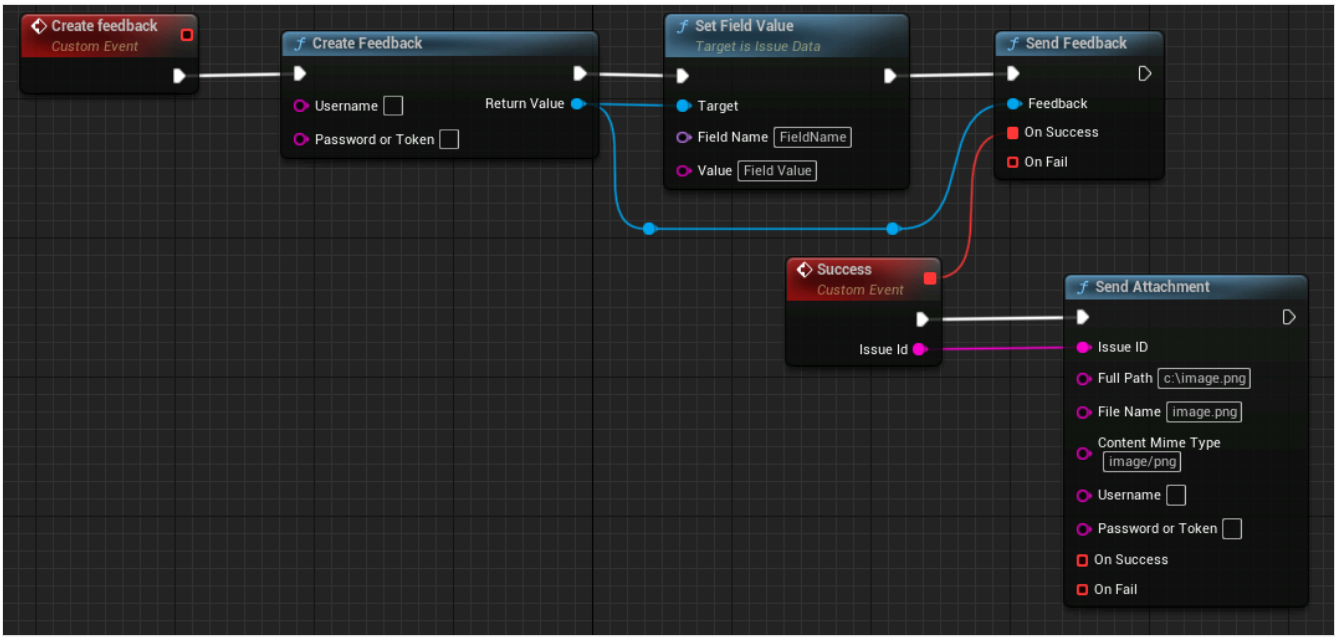
Example 2 - Report issue



Example 3 - Set fields value



Example 4 - Send user feedback with attachment



Example 5 - Send user feedback with attachment (async version)

The asynchronous nodes do the same as Example 4, with no custom events to wire up: each of them carries its own **On Success** and **On Failure** execution pins, so the whole graph is a single chain.

1. **Create Feedback Async** - **On Success** hands over the **Feedback** object.
2. **Set Field Value** - fill in what the user typed into the form.
3. **Send Feedback Async** - takes that **Feedback** object and, on success, returns the **Issue Id**.
4. **Send Attachment Async** - takes the **Issue Id** along with the path, file name and mime type of the file to upload.

Example 6 - Get field values